Como o Modelo Foi Treinado: Guia Completo

LightGBM, fine-tuning, few-shot/zero-shot, matriz de confusão e os 3 gráficos de entrega — explicado de forma objetiva

---

1. Estrutura do Modelo e Como Ele Treinou

1.1 — O que é o LightGBM

LightGBM é um algoritmo de **Gradient Boosted Decision Trees (GBDT)**: constrói centenas de pequenas árvores de decisão, uma a seguir à outra, onde cada árvore nova tenta corrigir os erros que as árvores anteriores ainda cometem. No final, a previsão é a soma ponderada de todas as árvores. É o mesmo princípio de família do XGBoost, mas com uma implementação mais rápida.

1.2 — Como treinou em Madrid (Stage 1)

Em vez de treinar 1 único modelo, o notebook treinou **5 modelos LightGBM independentes**, um por cada "fatia" (fold) dos dados de Madrid. A divisão dos dados NÃO foi aleatória — foi feita por **GroupKFold**, agrupando por célula espacial (px_key_py_key). Isto é uma escolha técnica importante:

Método	Como divide os dados	Risco
StratifiedKFold (usado no XGBoost/RF)	Divide pixel a pixel, mantendo a proporção de classes em cada fold	Pixels vizinhos (fisicamente próximos) podem cair um no treino e outro na validação — como são espacialmente correlacionados, o modelo "aprende" com um vizinho e é testado no outro, inflacionando artificialmente o F1
GroupKFold (usado no LightGBM)	Agrupa TODOS os pixels de uma mesma célula espacial no mesmo fold — nunca ficam divididos	Nenhum — é a divisão mais rigorosa, simula melhor o que acontece numa cidade nova, onde não há vizinhos "treinados" por perto

Além disso, 20% dos dados de Madrid foram separados como **"teste intocado"** — nunca usados nem no treino nem na validação cruzada, só no fim, para uma confirmação final e honesta de que o modelo generaliza (resultado: F1-macro = 0,63, accuracy = 65%).

1.3 — Os 5 modelos guardados

No final do treino em Madrid, os **5 modelos (um por fold)** são guardados em disco (madrid_models.pkl). Para o zero-shot em Amsterdam, as previsões dos 5 modelos são **calculadas em média** (um pequeno "comité de especialistas") — o que costuma ser mais estável do que confiar num único modelo.

2. Fine-Tuning do LightGBM e o Critério de Early Stopping

2.1 — O que é fine-tuning aqui

"Fine-tuning" significa **continuar a treinar** um modelo já existente, em vez de começar do zero. No código, isto é feito com o parâmetro init_model=base_model: cada um dos 5 modelos de Madrid é usado como ponto de partida, e o LightGBM adiciona **até 150 árvores novas**, ajustadas apenas com os poucos pixels rotulados de Amsterdam disponíveis (5 a 200 por classe). As árvores antigas (o conhecimento de Madrid) não são apagadas — só complementadas.

2.2 — O critério de early stopping (paragem antecipada)

"Early stopping" é uma técnica para evitar que o modelo **decore** os poucos exemplos de treino (overfitting): a cada nova árvore adicionada, o modelo verifica o seu erro num conjunto de validação separado; se o erro parar de melhorar durante 15 rondas seguidas (stopping_rounds=15), o treino para e fica-se com a melhor versão encontrada até ali.

O ponto crítico: o conjunto de validação usado para essa verificação é ams_val_data — que é o *fold de validação inteiro do GroupKFold em Amsterdam*, com cerca de **5.198 pixels rotulados**. Ou seja: o modelo treina os seus pesos só com os poucos exemplos "few-shot" (ex.: 25/classe = 100 pixels), mas decide QUANDO PARAR de treinar espreitando a performance em milhares de outros pixels rotulados de Amsterdam.

2.3 — Por que isto "infla ligeiramente os números de few-shot"

Pensa assim: se um aluno faz um exame usando só 5 exercícios para estudar, mas o professor deixa-o espreitar as respostas de 5.000 outros exercícios parecidos só para saber "quando parar de estudar" (não para copiar respostas diretamente, mas para saber se já está pronto) — o resultado final vai ser melhor do que se ele tivesse estudado às cegas, só com os 5 exercícios. Não é fraude (o modelo não usa os 5.198 rótulos para ajustar os pesos das árvores diretamente), mas usa essa informação extra para tomar uma decisão importante (quando parar), o que dá uma vantagem que um protocolo "estritamente N rótulos e mais nada" não teria. Por isso os números de few-shot do LightGBM (ex.: 0,620 com 25/classe) tendem a ficar ligeiramente mais altos do que ficariam num teste 100% justo — mas a diferença não é grande o suficiente para anular a vantagem geral do modelo.

3. Zero-Shot e Few-Shot, em Palavras Simples

Termo	O que significa	Analogia
Zero-shot	Aplicar o modelo treinado em Madrid diretamente em Amsterdam, SEM mostrar-lhe nenhum exemplo rotulado de Amsterdam antes	Um médico que nunca viu um doente Holandês, mas usa o que aprendeu a tratar doentes Espanhóis para tentar diagnosticar o primeiro caso Holandês
Few-shot	Mostrar ao modelo um PEQUENO número de exemplos rotulados de Amsterdam (5 a 200 por classe) antes de o avaliar, para ele se ajustar um pouco	O mesmo médico, mas agora já viu 25 casos reais de doentes Holandeses antes de continuar a diagnosticar os restantes

A diferença de desempenho entre os dois (zero-shot F1=0,511 vs. few-shot com 200/classe F1=0,682) mostra exatamente o valor de ter até um pouco de informação local — e é essa curva de melhoria que o gráfico da secção 6.3 visualiza.

4. A Estratégia de Transferência (Transfer Learning)

A estratégia tem 2 etapas, desenhadas para resolver o problema central do desafio: "como reconhecer idades de edifícios numa cidade nova, com o mínimo de rotulagem manual possível?"

Etapa 1 — Aprender o "vocabulário espectral" geral em Madrid

Treinar o modelo com muitos dados rotulados (76 mil pixels) para aprender como a refletância do satélite se relaciona com a idade de um edifício — um conhecimento que, em princípio, não é exclusivo de Madrid (materiais e padrões de envelhecimento têm semelhanças entre cidades).

Etapa 2 — Ajustar esse conhecimento a Amsterdam com poucos exemplos

Em vez de começar do zero (o que exigiria milhares de rótulos de Amsterdam, caros e demorados de obter), o modelo de Madrid serve de ponto de partida, e só precisa de uns poucos exemplos locais (fine-tuning) para se recalibrar às diferenças da nova cidade (clima, materiais, densidade urbana).

*É por isto que se chama "transfer learning" — conhecimento é **transferido** de um domínio (Madrid) para outro (Amsterdam), em vez de aprendido do zero em cada cidade nova.*

5. A Matriz de Confusão: O Que É e Para Que Serve

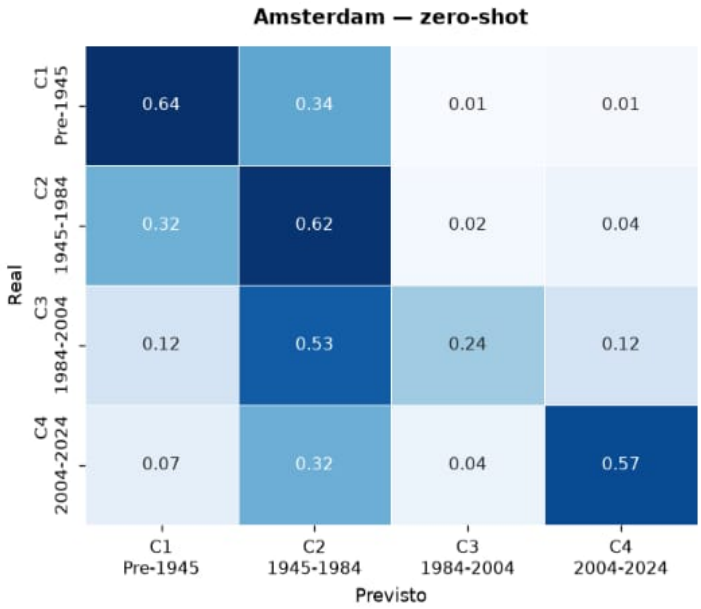
5.1 — O que é

A matriz de confusão é uma tabela 4x4 (porque há 4 classes) que compara o que o modelo **previu** com o que era **realmente verdade**, pixel a pixel. Não é um "mapa de correlação" no sentido estatístico tradicional (que mediria a relação entre duas variáveis numéricas) — é antes um **mapa de acertos e confusões entre categorias**.

5.2 — Para que serve

Uma métrica única como "macro-F1 = 0,51" diz-te QUE TÃO BOM é o modelo em média, mas não diz ONDE ele erra. A matriz de confusão mostra exatamente isso — **que classes são confundidas com que outras classes**, o que é essencial para diagnosticar a causa dos erros e decidir o que corrigir.

5.3 — Como ler a matriz que enviaste (Amsterdam zero-shot)



Cada célula (linha, coluna) mostra: **"de todos os pixels que SÃO REALMENTE da classe da linha, que fração o modelo PREVIU como sendo da classe da coluna"**. Por isso, cada linha soma sempre 1,00 (ou 100%).

- **Diagonal principal (C1→C1=0,64, C2→C2=0,62, C3→C3=0,24, C4→C4=0,57):** são os acertos. Quanto mais escura e mais perto de 1,00, melhor. Repara que a Classe 3 (1984-2004) só acerta 24% das vezes — é o ponto mais fraco do modelo nesta versão.
- **Fora da diagonal:** são os erros, e a CÉLULA em si diz para onde o erro vai. Por exemplo, "C3 → C2 = 0,53" significa que 53% dos pixels que são realmente da Classe 3 (1984-2004) foram classificados (erradamente) como Classe 2 (1945-1984) — o maior erro isolado de toda a matriz.
- **Padrão geral:** os erros tendem a concentrar-se nas classes vizinhas no tempo (C1↔C2, C3↔C2), quase nunca entre classes muito distantes (C1↔C4 é sempre próximo de 0,01-0,07) — o que faz sentido: um edifício de 1950 tem mais chance de ser confundido com um de 1970 do que com um de 2020.

6. Os 3 Gráficos/Tabelas Importantes, Explicados

6.1 — Matriz de confusão

O que é: tabela 4x4 que mostra onde o modelo acerta e erra entre as 4 classes (explicado em detalhe na secção 5). **Para que serve na entrega:** permite ao júri do hackathon ver, de relance, se os erros do teu modelo fazem sentido (confusão entre classes vizinhas no tempo) ou se há um problema estrutural mais grave (ex.: confundir sistematicamente as duas classes mais distantes).

6.2 — Tabela: F1-score de Madrid vs. F1-score de Amsterdam

O que é: uma tabela com 1 linha para o F1 de Madrid (o "baseline", medido por validação cruzada) e várias linhas para o F1 de Amsterdam, uma por cada tamanho de amostra usado no few-shot (5, 10, 25, 50, 100, 200 pixels/classe), cada uma com a média ± desvio-padrão (as "barras de erro"). **Para que serve:** é a prova numérica de que a estratégia de transferência funciona — mostra concretamente quanto esforço de rotulagem (nº de amostras) é preciso para uma cidade nova "apanhar" o desempenho da cidade original.

6.3 — Gráfico: F1 vs. log2(tamanho da amostra)

O que é: o mesmo conteúdo da tabela anterior, mas desenhado como um gráfico de linha, com o eixo X em escala **log2** (logaritmo de base 2) do número de amostras, e o eixo Y o macro-F1, com barras de erro verticais. **Por que log2 e não o número normal:** os tamanhos de amostra testados (5, 10, 25, 50, 100, 200) duplicam ou quase duplicam a cada passo — numa escala normal, os primeiros pontos ficariam todos esmagados à esquerda do gráfico e os últimos muito espalhados à direita. Em escala log2, cada "duplicação" de amostras ocupa a mesma distância visual no eixo X, tornando fácil ver se o ganho de F1 por cada duplicação de dados está a acelerar, estabilizar ou já saturou. **Para que serve:** é o gráfico mais importante da apresentação — mostra visualmente a curva de aprendizagem da adaptação e onde está o "ponto de equilíbrio" ideal entre esforço de rotulagem e ganho de precisão (no nosso caso, por volta de 25-50 amostras/classe, onde a curva já cruza a linha de Madrid).

7. Ligação com a Análise Exploratória de Dados (EDA) — Passo a Passo

Estes 3 gráficos de modelagem não nascem isolados — são a continuação direta de decisões tomadas na EDA (Notebook 2). Eis a ligação:

→ **EDA identificou desequilíbrio de classes:** Classe 1/2 dominam Madrid (18%/36%) → motivou o uso de class_weight/sample_weight nos modelos e ajuda a explicar por que a Classe 2 é sobre-prevista na matriz de confusão (secção 5).

→ **EDA confirmou correlação espacial entre pixels vizinhos:** → motivou diretamente a escolha do GroupKFold em vez de StratifiedKFold no LightGBM (secção 1.2), evitando uma validação artificialmente otimista.

→ **EDA mostrou que NIR/SWIR trazem mais informação que RGB:** → confirma-se na importância de atributos dos modelos treinados (NIR_early_mean lidera em todos eles), validando a decisão de usar as 6 bandas completas.

→ **EDA revelou que a refletância média/variabilidade por pixel discrimina classes:** → é exatamente isso que as 60 features de entrada (médias e desvios-padrão por período) tentam capturar numericamente para o modelo.

→ **EDA já alertava para a ambiguidade da Classe 3:** (construção durante o próprio período de observação do satélite) → explica diretamente por que a Classe 3 é a mais fraca em todas as matrizes de confusão vistas até agora.

8. Conclusão e Limitações do Modelo

Conclusão: o LightGBM com fine-tuning incremental é o modelo mais forte testado até agora (avg F1 0,604, peak F1 0,682), com a validação de Madrid mais rigorosa (GroupKFold espacial + teste intocado) e uma estratégia de transferência que efetivamente melhora com mais dados locais, em vez de saturar cedo como o protótipo simples.

Limitações a reconhecer com transparência:

- **Early stopping com acesso indireto a muitos rótulos** (secção 2.3) — os números de few-shot não seguem um protocolo 100% "N rótulos e nada mais"; um avaliador externo rigoroso poderia pedir uma reavaliação com early stopping baseado apenas nos próprios poucos exemplos few-shot.
- **Classe 3 continua a ser o ponto fraco estrutural** — a ambiguidade de dados (o próprio período de construção sobrepõe-se à janela de observação usada como feature) não é resolvida só com um modelo melhor; precisaria de nova engenharia de atributos dedicada a essa classe.
- **Sem teste intocado em Amsterdam** — ao contrário de Madrid (que tem 20% de teste nunca tocado), a avaliação em Amsterdam usa sempre o resto do fold do GroupKFold como validação, sem uma reserva final 100% independente.
- **Custo computacional** — 5 modelos GBDT + fine-tuning por orçamento de amostra é mais pesado de treinar e manter do que uma abordagem única (ex.: só um classificador + protótipo), o que pode ser relevante se o sistema precisar de escalar para dezenas de cidades no futuro.